



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Дајана Булић

ИМПЛЕМЕНТАЦИЈА ОБСЕРВАБИЛНОСТИ МИКРОСЕРВИСНЕ АРХИТЕКТУРЕ

ЗАВРШНИ РАД
- Основне академске студије -

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :			
Идентификациони број, ИБР :			
Тип документације, ТД :		Монографска документација	
Тип записа, ТЗ :		Текстуални штампани материјал	
Врста рада, ВР :		Завршни (Bachelor) рад	
Аутор, АУ :		Дајана Булић	
Ментор, МН :		др Себастијан Стоја, доцент	
Наслов рада, НР :		Имплементација обсервабилности микросервисне архитектуре	
Језик публикације, ЈП :		Српски/ ћирилица	
Језик извода, ЈИ :		Српски	
Земља публиковања, ЗП :		Република Србија	
Уже географско подручје, УГП :		Војводина	
Година, ГО :		2026	
Издавач, ИЗ :		Ауторски репринт	
Место и адреса, МА :		Нови Сад, Трг Доситеја Обрадовића 6	
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)		7 поглавља / 19 страна / 10 цитата / 3 табеле / 2 слике / 0 графика / 0 прилога	
Научна област, НО :		Електротехничко и рачунарско инжењерство	
Научна дисциплина, НД :		Примењено софтверско инжењерство	
Предметна одредница/Кључне речи, ПО :		обсервабилност, микросервисна архитектура, OpenTelemetry, дистрибуирано праћење захтева (distributed tracing), структурисано	
УДК			
Чува се, ЧУ :		У библиотеци Факултета Техничких Наука	
Важна напомена, ВН :			
Извод, ИЗ :		У овом раду анализиран је постојећи пројекат Travel Planner (микросервисна апликација за планирање путовања, реализована на Microsoft Service Fabric платформи) са аспекта обсервабилности, идентификовани су недостаци постојећег решења (одсуство структурисаног логовања, метрика, дистрибуираног праћења захтева и health check механизма), и предложена је и имплементирана надоградња заснована на OpenTelemetry стандарду. Решење је верификовано кроз аутоматизоване тестове и кроз демонстрационе сценарије извршене над стварно постављеним системом.	
Датум прихватања теме, ДП :			
Датум одбране, ДО :			
Чланови комисије, КО :	Председни		
Члан:			Потпис ментора
Члан, ментор:		др Себастијан Стоја, доцент	



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :		Monographic publication	
Type of record, TR :		Textual printed material	
Contents code, CC :		Bachelor Thesis	
Author, AU :		Dajana Bulic	
Mentor, MN :		Sebastijan Stoja, PhD, Assistant professor	
Title, TI :		Implementation of Observability in a Microservice Architecture	
Language of text, LT :		Serbian	
Language of abstract, LA :		Serbian/English	
Country of publication, CP :		Serbia	
Locality of publication, LP :		Vojvodina	
Publication year, PY :		2026	
Publisher, PB :		Author reprint	
Publication place, PP :		Faculty of Tehnical Sciences, D. Obradovića 6, 2100 Novi Sad	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)		7 chapters / 19 pages / 10 references / 3 tables / 2 figures / 0 graphs / 0 appendixes	
Scientific field, SF :		Electrical and computer engineering	
Scientific discipline, SD :		Power Software Engineering	
Subject/Key words, S/KW :		observability, microservice architecture, OpenTelemetry, distributed tracing, structured logging, metrics, Prometheus, Grafana, health checks, Service	
UC			
Holding data, HD :		Library of the Faculty of Tehnical Sciences	
Note, N :			
Abstract, AB :		This thesis analyzes an existing project, Travel Planner (a microservice-based travel planning application built on the Microsoft Service Fabric platform), from an observability standpoint, identifies the shortcomings of the existing solution (absence of structured logging, metrics, distributed tracing and health check mechanisms), and proposes and implements an upgrade based on the OpenTelemetry standard and a Prometheus/Grafana Tempo/Grafana Loki/Grafana stack. The solution is verified through automated tests and through demonstration scenarios executed against the actually deployed system.	
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended Board, DB :	President:		
	Member:		Menthor's sign
	Member, Mentor:	PhD Sebastijan Stoja, assistant professor	

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Примењено софтверско инжењерство		
Студент:	Дајана Булић	Број индекса:	ПР2-2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	др Себастијан Стоја, доцент		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Имплементација обсервабилности микросервисне архитектуре

ТЕКСТ ЗАДАТКА:

Анализирати постојећи пројекат Travel Planner (микросервисна веб апликација за планирање путовања: Gateway, AuthService, TripService, SharingService, Microsoft Service Fabric, Microsoft SQL Server) са аспекта обсервабилности мониторинга и управљања микросервисном архитектуром. Идентификовати недостатке постојећег решења у погледу логовања, метрика, дистрибуираног праћења захтева (distributed tracing) и health check механизма. Предложити и имплементирати надоградњу засновану на OpenTelemetry стандарду, укључујући прикупљање и визуелизацију телеметријских података (Prometheus, Grafana Tempo, Grafana Loki, Grafana). Резултате рада верификовати кроз аутоматизоване тестове и кроз демонстрационе сценарије (нормалан захтев, грешка, повећано оптерећење, спора зависност) над стварно постављеним системом.

Руководилац студијског програма:	Ментор рада:

Примерак за: О - Студента; О - Ментора



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

Списак слика.....	1
Списак табела.....	2
Списак скраћеница.....	3
1. Увод.....	4
2. Опис проблема.....	5
3. Опис коришћених технологија и алата.....	6
4. Опис почетног решења.....	8
5. Опис решења проблема.....	10
6. Тестирање и резултати.....	12
7. Закључак.....	15
Литература.....	16
Биографија.....	17

Списак слика

Слика 4.1. Архитектура система пре надоградње	8
Слика 5.1. ТО-ВЕ архитектура — додат observability слој	10

Списак табела

Табела 6.1. Резултати аутоматизованог тестирања (dotnet test)	12
Табела 6.2. Поређење система пре и после надоградње	14

Списак скраћеница

Скраћеница	Значење
API	Application Programming Interface (програмски интерфејс за приступ функционалностима)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција и испорука)
CPU	Central Processing Unit (централна процесорска јединица)
EF Core	Entity Framework Core (ORM оквир коришћен за приступ бази података)
HTTP	HyperText Transfer Protocol (протокол преноса хипертекста)
IEEE	Institute of Electrical and Electronics Engineers (стандард цитирања коришћен у раду)
JWT	JSON Web Token (формат токена за аутентикацију)
OTel	OpenTelemetry (стандард обсервабилности коришћен у раду)
OTLP	OpenTelemetry Protocol (протокол преноса телеметрије ка Collector-у)
RED	Rate, Errors, Duration (стандардна група метрика: стопа захтева, стопа грешака, трајање)
REST	Representational State Transfer (архитектурни стил веб API-ја)
SDK	Software Development Kit (скуп алата и библиотека за развој)
SF	Service Fabric (Microsoft-ова платформа за микросервисну оркестрацију)
SQL	Structured Query Language (језик за рад са релационим базама података)
W3C	World Wide Web Consortium (стандард за пропагацију trace контекста, transparent)
YARP	Yet Another Reverse Proxy (reverse proxy библиотека коришћена у Gateway-у)

1. Увод

Travel Planner је микросервисна веб апликација за планирање путовања која кориснику омогућава да на једном месту организује план путовања: основне податке о путовању, дестинације, дневне активности (са приказом кроз календар), трошкове и буџет, белешке, подсетнике, checklist/packing листу, као и дељење плана путем кода или QR кода са другим особама. Овај рад се бави имплементацијом обсервабилности над постојећим системом Travel Planner.

[ПОПУНИ: убацити слику 1.1 — screenshot почетне/главне стране Travel Planner апликације (нпр. преглед плана путовања са дестинацијама и календаром), ради визуелног увода читаоца у изглед и функционалност апликације пре описа архитектуре.]

[ПОПУНИ: убацити слику 1.2 — screenshot екрана за дељење плана путовања (генерисани код и/или QR код), ради приказа SharingService функционалности описане у поглављу 4.]

Систем је реализован као микросервисна архитектура на платформи Microsoft Service Fabric, са четири одвојена сервиса (Gateway, AuthService, TripService, SharingService) и базом података по сервису (database-per-service) у Microsoft SQL Server-у. Оваква архитектура доноси познате предности микросервиса — независно скалирање, одвојен развој и deployment по сервису — али и познат проблем: један корисников захтев често пролази кроз више процеса, база и мрежних позива, па постаје знатно теже одговорити на питања попут „зашто је овај захтев спор?“, „где тачно долази до грешке?“ или „да ли је систем тренутно здрав?“ него код монолитне апликације.

Тема овог рада, Имплементација обсервабилности микросервисне архитектуре, директно адресира тај проблем. Циљ рада није да мења пословну логику постојећег система (планове, дестинације, трошкове), већ да га надогради слојем обсервабилности заснованим на индустријском стандарду OpenTelemetry — дистрибуирано праћење захтева (distributed tracing), структурисано логовање, метрике перформанси и health check механизме — и да ту надоградњу докаже кроз стварно извршавање, аутоматизоване тестове и мерљиве резултате, а не само кроз инсталацију алата.

Рад је организован тако да прво анализира постојеће стање система (поглавља 2-4), затим детаљно описује предложено и имплементирано решење (поглавље 5), верификује га кроз тестирање и демонстрационе сценарије над стварно постављеним системом (поглавље 6), и на крају сумира постигнуте резултате, ограничења и правце даљег развоја (поглавље 7).

2. Опис проблема

Анализа постојећег система (детаљно описаног у поглављу 4) показала је да Travel Planner, иако функционално комплетан и добро структурисан у погледу поделе одговорности између сервиса, нема ниједан механизам обсервабилности изнад апсолутног минимума који сам ASP.NET Core оквир пружа подразумевано. Конкретно, идентификовани су следећи проблеми:

Немогуће праћење захтева кроз више сервиса

Захтев корисника типично пролази кроз ланац Frontend → Gateway → сервис (нпр. AuthService), а поједини токови иду и даље, сервис-сервис (нпр. SharingService → TripService приликом разрешавања дељеног плана). Не постоји ниједан идентификатор (correlation/trace ID) који би повезао логове настале у различитим процесима за исти кориснички захтев. Ако захтев успори или падне негде у ланцу, једини начин да се проблем лоцира био би ручно упоређивање временских печата логова из различитих сервиса — непоуздано и споро.

Одсуство метрика перформанси

Ниједан сервис не излаже метрике броја захтева, латенције, стопе грешака нити искоришћења ресурса (CPU, меморија). Пад перформанси (нпр. спор упит ка бази) остао би потпуно невидљив док се корисници директно не пожале.

Тихе грешке

У коду постоје примери у којима се грешка ухвати (try/catch) и обради без иједног трага у логу. Конкретан пример: AuthService/Services/TripClient.cs, метода DeleteUserTripsAsync — при брисању корисничког налога позива се TripService да обрише све планове тог корисника; ако тај позив не успе (сервис недоступан, мрежни проблем), изворни код је само враћао false, без иједног записа у логу. Систем би остао недоследан (обрисан налог, а планови и даље постоје у бази), без иједног трага зашто.

Одсуство health check механизма

Ниједан сервис не излаже endpoint преко ког би се споља (оркестрација, monitoring алат, човек) проверило да ли сервис стварно ради и да ли су његове зависности (база, други сервиси) доступне. Service Fabric поседује сопствени интерни health модел, али му се health стање апликационог нивоа (нпр. „да ли TripService може да се повеже на своју базу“) никада не пријављује.

Нестандардизовано, минимално логовање

Свака appsettings.json конфигурација дефинише ниво логовања (Information/Warning), али у пракси се у коду скоро нигде не позива ILogger — grep кроз цео backend је показао 0 позива ILogger/_logger пре овог рада. Постојећи Service Fabric ETW ServiceEventSource механизам користи се искључиво за lifecycle догађаје (покретање Kestrel-а, регистрација сервисног типа), не за токове пословне логике.

Ови проблеми су директно повезани — сви произилазе из недостатка јединственог, стандардизованог слоја обсервабилности, а не из појединачних, изолованих пропуста. То је управо разлог зашто је предложено решење (поглавље 5) целовит слој заснован на једном стандарду (OpenTelemetry), а не низ ad-hoc закрпа.

3. Опис коришћених технологија и алата

Постојеће технологије (backend/frontend)

Backend четири сервиса (Gateway, AuthService, TripService, SharingService) реализован је у C#/NET 8, хостован на Microsoft Service Fabric платформи [4] (stateless сервиси за Gateway/AuthService/TripService, stateful за SharingService), уз Entity Framework Core 8 за приступ подацима, YARP (Yet Another Reverse Proxy) [5] за Gateway рутирање, BCrypt.Net за хеширање лозинки и JWT Bearer аутентикацију. Подаци се чувају у Microsoft SQL Server бази, по принципу database-per-service (UsersDB, TripsDB, SharingDB). Frontend је React 19 (Vite) апликација која комуницира искључиво преко Gateway-а.

У контексту Service Fabric платформе, stateless сервис не чува стање између позива на самом чвору — сваки захтев се може опслужити било којом инстанцом, а трајно стање се чува искључиво у спољној бази (SQL Server). Stateful сервис, насупрот томе, поседује локално, реплицирано стање непосредно на Service Fabric чворовима (Reliable Collections), поред спољне базе — тако да поједини подаци могу да се читају без мрежног позива ка бази. У овом пројекту, три сервиса (Gateway, AuthService, TripService) су stateless, док је SharingService stateful — детаљније објашњено у поглављу 4.

OpenTelemetry

OpenTelemetry (OTel) изабран је као централни стандард надоградње јер обједињује сва три облика телеметрије (traces, metrics, logs) под једним, vendor-неутралним API-јем и SDK-ом [2], са великим бројем готових ("auto") инструментација управо за технологије које пројекат већ користи — ASP.NET Core, HttpClient, SqlClient — тако да се велик део вредности добија без писања сопственог инструментационог кода. Верзија OpenTelemetry .NET SDK-а коришћена у раду је 1.18.0, конзистентна кроз сва четири сервиса.

Алтернативе попут Application Insights-а или vendor-специфичних APM решења (нпр. Datadog, New Relic) су разматране, али одбачене управо због vendor lock-in-а — OTel омогућава промену backend-а за складиштење (Prometheus/Tempo/Loki → неко друго решење) без измене иједног сервиса, пошто је инструментациони код одвојен од места где телеметрија завршава.

OpenTelemetry Collector

Сервиси не шаљу телеметрију директно ка Prometheus/Tempo/Loki, већ ка једном централном OpenTelemetry Collector-у (OTLP протокол, gRPC на порту 4317). Ово раздваја „како апликација емитује телеметрију” од „где та телеметрија завршава” — каснија измена backend-а за складиштење (нпр. замена Loki-ја другим решењем) не би захтевала измену ниједног сервиса, само Collector-ове конфигурације [10].

Prometheus, Grafana Tempo, Grafana Loki

За складиштење три врсте телеметрије изабран је такозвани „Grafana stack”: Prometheus [9] за метрике (Collector излаже Prometheus-компатибилан /metrics endpoint који Prometheus периодично scrape-ује), Grafana Tempo [6] за дистрибуирано праћење захтева (trace-ове), и Grafana Loki [7] за централизоване логове. Предност ове комбинације над, на пример, самостојећим Jaeger-ом за tracing јесте што све три компоненте имају нативну, provisioned интеграцију са Grafanom — укључујући унакрсну навигацију (клик са log линије на њен trace, клик са trace-а на метрику) која је директно демонстрирана у поглављу 5.

Grafana

Grafana је изабрана као јединствен визуелни слој за све три врсте телеметрије, уместо по једног специјализованог алата за сваку (нпр. Jaeger UI за трасе-ове, посебан алат за метрике). Ово директно подржава сценарио „развијач види комплетну слику једног захтева на једном месту”, који је централна теза рада.

Health checks

Коришћен је уграђен ASP.NET Core механизам (Microsoft.Extensions.Diagnostics.HealthChecks), допуњен пакетом AspNetCore.HealthChecks.SqlServer (верзија 9.0.0, компатибилна са .NET 8) за проверу доступности SQL Server базе по сервису [3].

k6

За демонстрацију понашања система под оптерећењем (сценарио 3, поглавље 5) коришћен је k6 (Grafana k6), алат за load testing чији се скриптовани сценарији пишу у JavaScript-у, што га чини лакшим за одржавање и читање у односу на алтернативе попут JMeter-a [8].

xUnit

За аутоматизовано тестирање (поглавље 6) коришћен је xUnit, стандардни test framework за .NET, јер пројекат пре овог рада није имао дефинисан test framework нити иједан тест.

Docker / Docker Compose

Цела observability инфраструктура (Collector, Prometheus, Tempo, Loki, Grafana) покреће се кроз Docker Compose, независно од постојећег Service Fabric кластера и SQL Server инстанце — додатак, не замена постојећег начина рада система.

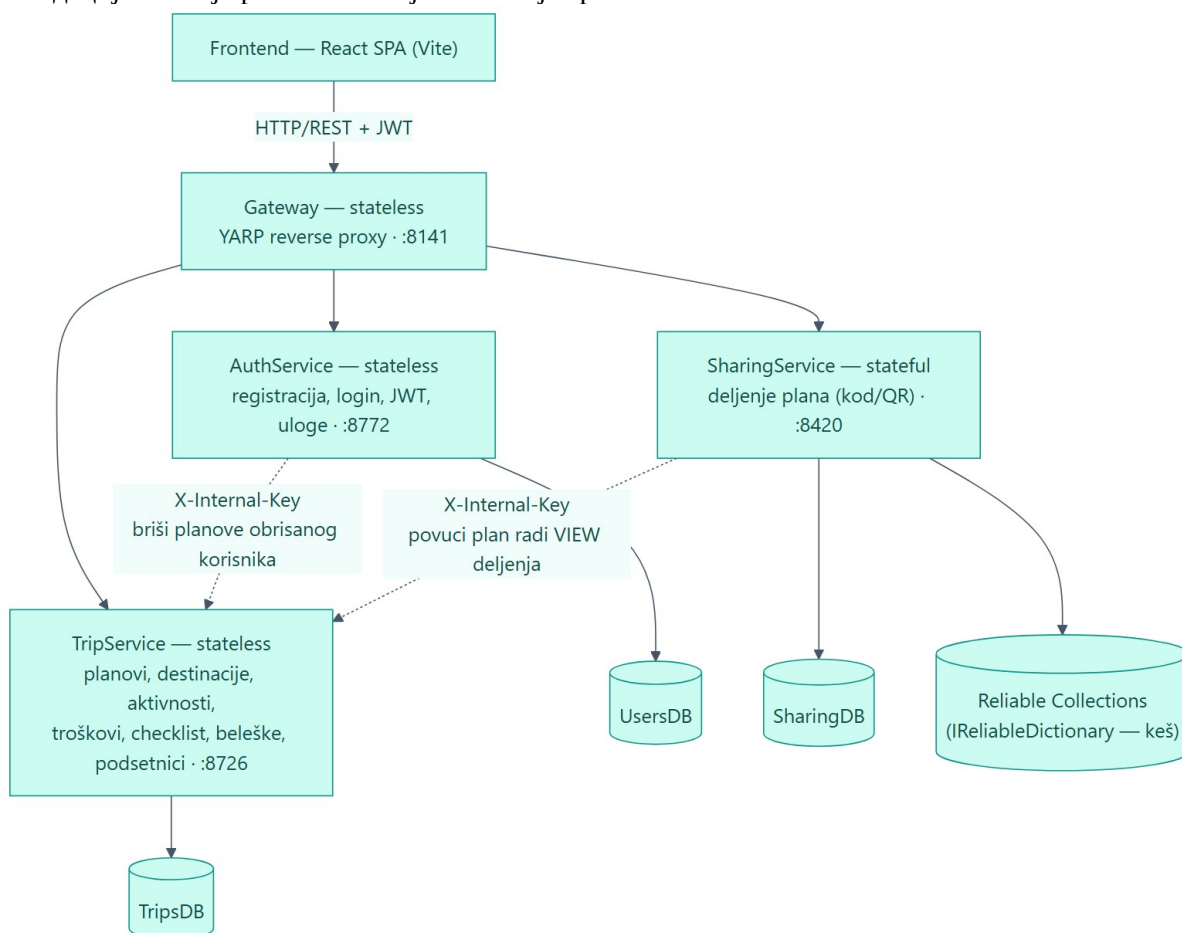
4. Опис почетног решења

У овом поглављу описано је стање система Travel Planner пре надоградње описане у овом раду — архитектура, технологије и, посебно, постојећи (или тачније, непостојећи) механизми обсервабилности, установљени детаљном анализом изворног кода.

Архитектура система

Систем чине четири одвојена сервиса на Microsoft Service Fabric платформи:

- Gateway (stateless) — YARP reverse proxy, једина улазна тачка за frontend; рутира захтеве ка осталим сервисима по путањи, без сопствене пословне логике или базе.
- AuthService (stateless) — регистрација, пријава, издавање и потписивање JWT токена, улоге (Корисник/Admin), администрација корисничких налога. База UsersDB.
- TripService (stateless) — централни сервис са целокупним садржајем плана: дестинације, активности, трошкови/буџет, checklist, белешке, подсетници. База TripsDB.
- SharingService (stateful) — дељење плана путем кода/QR-а (VIEW/EDIT). SharingDB је извор истине, а важећи кодови се додатно кеширају у Service Fabric Reliable Collections ради брже валидације — то је разлог зашто је баш овај сервис stateful.



Слика 4.1. Архитектура система пре надоградње

[ПОПУНИ: убацили слику 4.2 — screenshot Service Fabric Explorer-a (localhost:19080) са приказаним fabric:/TravelPlanner стаблом, сва 4 сервиса зелена (Healthy/Ready), као доказ да AS-IS архитектура описана изнад стварно постоји и ради на локалном кластеру.]

Комуникација frontend-а са системом иде искључиво преко Gateway-а (HTTP/REST + JWT). Позиви сервис-сервис (AuthService → TripService приликом брисања налога, SharingService → TripService приликом VIEW приступа) иду директно, мимо Gateway-а, и осигурани су дељеним

тајним кључем у X-Internal-Key заглављу — не JWT токеном, јер у том контексту не постоји прављени корисник.

Избор stateless дизајна за Gateway/AuthService/TripService последица је чињенице да ниједан од њих не мора да памти стање између захтева — сав неопходан state већ постоји у бази. SharingService је једини stateful сервис јер важећи кодови за дељење морају бити доступни са ниском латенцијом при сваком разрешавању (VIEW приступ анонимног посетиоца), па се кеширају директно на Service Fabric чвору уместо да се за сваки захтев чита из SharingDB базе.

Постојећи механизми обсервабилности

Детаљна анализа изворног кода (претрага на ILogger/_logger позиве, health check регистрације, tracing/metrics пакете) показала је следеће затечено стање:

- Логовање: искључиво подразумевани ASP.NET Core Console logger (ниво Information/Warning дефинисан у appsettings.json). Претрага кроз цео backend показала је нула позива ILogger/_logger — контролери, сервиси и HTTP клијенти не логују ништа.
- Service Fabric ETW ServiceEventSource постоји по сервису, али се користи искључиво за lifecycle догађаје (покретање Kestrel-а, регистрација сервисног типа) — методе ServiceRequestStart/Stop су дефинисане, али се нигде не позивају.
- Метрике: не постоје — нема Prometheus exporter-а нити коришћења .NET Metrics API-ја.
- Дистрибуирано праћење захтева: не постоји. Нема trace/correlation ID-а нигде у систему — ни на frontend-у, ни на Gateway-у, ни у сервис-сервис позивима.
- Health checks: AddHealthChecks() се нигде не позива — ниједан сервис нема /health endpoint. Service Fabric-у се health стање апликационог нивоа никада не пријављује.
- Руковање грешкама: нема глобалног exception middleware-а; неухваћени изузеци падају на ASP.NET Core подразумевано понашање, без структурисаног лога.
- Тестови: не постоји ниједан тест пројекат, ни backend ни frontend.

Ово затечено стање и конкретни проблеми који из њега произилазе детаљно су описани у поглављу 2 и представљају директну полазну тачку за решење описано у поглављу 5.

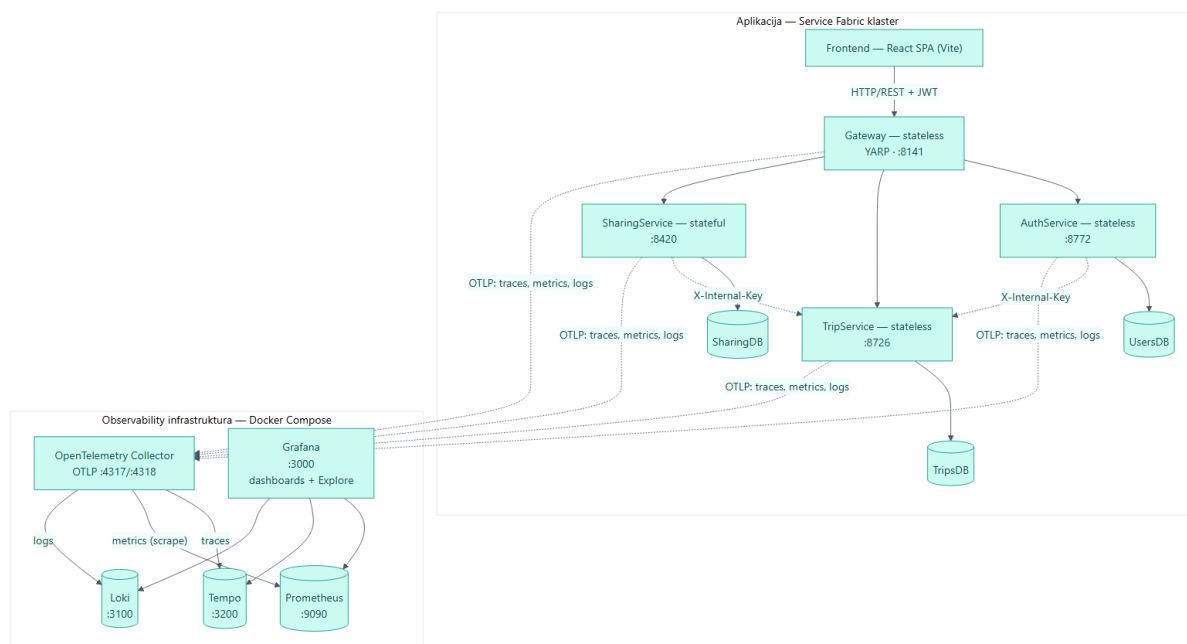
5. Опис решења проблема

Решење је реализовано као додатни слој обсервабилности изнад постојећег система, без измене пословне логике. Имплементација је спроведена кроз шест логичких целина: health checks, инфраструктура за прикупљање телеметрије, инструментација сервиса (traces/metrics/logs), Grafana dashboard, тестирање (описано одвојено у поглављу 6) и демонстрациони сценарији над стварно постављеним системом.

ТО-ВЕ архитектура

ТО-ВЕ ("to be" — стање које треба да буде) означава циљну архитектуру система НАКОН надоградње описане у овом раду, за разлику од AS-IS архитектуре (поглавље 4), која описује стање ПРЕ надоградње. Разлика између ове две архитектуре је управо додати observability слој приказан на слици 5.1.

Свих четири сервиса шаљу телеметрију преко OTLP протокола ка једном OpenTelemetry Collector-у, који је даље усмерава: трасе-ове ка Tempo-у, метрике ка Prometheus-у (преко сопственог /metrics endpoint-а који Prometheus периодично scrape-ује), логове ка Loki-ју. Grafana приказује сва три извора обједињено, укључујући унакрсну навигацију (log линија → trace, trace → метрика).



Слика 5.1. ТО-ВЕ архитектура — додат observability слој

Пропагација трасе контекста кроз сервис-сервис позиве (нпр. SharingService → TripService) дешава се аутоматски, без иједне линије custom кода: пошто и позивалац и прималац користе исту OpenTelemetry HttpClient/ASP.NET Core инструментацију, W3C tracerparent заглавље се чита из долазног захтева и пише на одлазни у оквиру истог ambient Activity контекста. X-Internal-Key заглавље остаје непромењено (механизам аутентикације), tracerparent је одвојено, стандардно заглавље.

Health checks

Сваки сервис излаже два endpoint-а: /health/live (процес је жив, не додирује зависности — намењен одлукама о рестарту) и /health/ready (процес и критичне зависности спремни да опслуже саобраћај). За AuthService/TripService/SharingService, /health/ready проверава доступност сопствене SQL Server базе. Gateway нема сопствену базу — његов /health/ready проверава мрежну

доступност три downstream сервиса, преко истих адреса које YARP већ користи за рутирање (без дуплирања конфигурације). Одговор је структурисан JSON са статусом сваке појединачне провере и, у случају грешке, читљивом поруком.

[ПОПУНИ: убацити слику 5.4 — screenshot browser-a са отвореним `http://localhost:8141/health/ready`, приказан сирови JSON одговор са статусом сва три downstream сервиса, ради доказа да је структурисан одговор описан изнад стваран.]

Инструментација сервиса

Заједничка OpenTelemetry конфигурација (traces + metrics + logs) дефинисана је у по једној ObservabilityExtensions класи по сервису (исти образац поновљен по сервису, јер пројекат нема дељену библиотеку између сервиса), и укључује:

- Traces — ASP.NET Core инструментацију (server span по захтеву), HttpClient инструментацију (пропагација контекста кроз сервис-сервис позиве, описано изнад) и SqlClient инструментацију (child span по SQL упиту, на три сервиса са базом). Намерно без снимања сировог SQL текста — у коришћеној верзији пакета то захтева експлицитно укључивање преко environment променљиве управо због ризика цурења података кроз SQL параметре; уместо тога користи се RecordException.
- Metrics — RED метрике (request rate, error rate, latency) из ASP.NET Core/HttpClient инструментације, плус метрике извршног окружења (garbage collector, thread pool, JIT, изузеци) из OpenTelemetry.Instrumentation.Runtime пакета — намерно без CPU/меморија OS нивоа, за то би био потребан засебан OpenTelemetry.Instrumentation.Process пакет који у тренутку писања рада нема стабилно (не-prerelease) издање — све без иједне линије custom кода.
- Logs — ILogger и даље пише на конзолу (непромењено, ради локалног развоја), а сада додатно и преко OpenTelemetry logging провајдера ка Loki-ју, са аутоматски додатим trace_id/span_id и ASP.NET Core контекстом (путања захтева, connection ID) у сваком запису.

Као директна последица овог рада, попуњен је и конкретан „тих” catch блок пронађен приликом анализе (AuthService/Services/TripClient.cs, метода DeleteUserTripsAsync) — неуспешан HTTP статус сада производи LogWarning, мрежни изузетак производи LogError, уместо да се грешка потпуно прогута.

Grafana dashboard

Provisioned dashboard „Travel Planner - Overview” приказује RED метрике (request rate, error rate, p95 latency), ресурсе (меморија/CPU/GC) и ток логова, све филтрирано по сервису преко template варијабле. Datasource-и (Prometheus/Tempo/Loki) су такође provisioned, укључујући derived field везу која омогућава да се из log линије у Loki-ју једним кликом отвори одговарајући trace у Tempo-у.

[ПОПУНИ: убацити слику 5.2 — screenshot Grafana dashboard-a „Travel Planner - Overview” dok је sistem pokrenut, sa vidljivim RED metrikama, resursima i log stream panelom.]

[ПОПУНИ: убацити слику 5.3 — screenshot Grafana Explore -> Tempo -> waterfall приказ trace-a из Сценарија 1 (Gateway -> SharingService -> TripService, са видљивих 9 SQL child span-ова), ради визуелног доказа дистрибуираног праћења захтева описаног изнад.]

6. Тестирање и резултати

Формално тестирање намерно је ограничено на оно што је овај рад заиста додао — health checks и observability pipeline. Постојећа пословна логика (планирање, дестинације, трошкови) није имала тестове пре овог рада и није предмет теме; проширивање обима тамо било би ван онога што је затражено.

Рефакторинг ради тестабилности

Пре писања тестова, DI регистрација и middleware pipeline сваког сервиса издвојени су из анонимне lambda функције унутар Service Fabric listener factory-ја у јавне статичке методе `ConfigureServices(WebApplicationBuilder)` и `ConfigurePipeline(WebApplication)` (по једна класа по сервису: `GatewayApp`, `AuthServiceApp`, `TripServiceApp`, `SharingServiceApp`). Service Fabric и даље креира `WebApplicationBuilder` и конфигурише `WebHost` (потребан му је `listener/url` из SF контекста), али саму регистрацију сервиса и middleware pipeline сада позива из ове заједничке методе — идентично без обзира да ли позивалац сервиса хостује SF или тест. Рефакторинг је чисто издвајање, без промене понашања, што је потврђено поновним build-овањем сва четири сервиса и redeploy-ем на Service Fabric кластер, уз проверу да су сви `/health/live` и `/health/ready` endpoint-и остали идентично здрави пре и после.

Тест пројекти и стратегија

Четири xUnit пројекта (по један за сваки сервис) не користе `WebApplicationFactory` (Service Fabric hosting модел нема класичну Program улазну тачку потребну за то) — сваки тест подиже прав Kestrel host на ефемерном порту преко издвојених `ConfigureServices/ConfigurePipeline` метода, без mock-а базе података.

- `HealthEndpointsTests` (сва четири сервиса) — интеграциони тестови, прав HTTP позив ка правом покренутом host-у. За `AuthService/TripService/SharingService` користи се права локална SQL Server база. `Gateway` користи `FakeDownstreamServer` (минималан прав HTTP сервер покренут у тест процесу) да симулира доступан/недоступан downstream сервис. Покривено за сва четири сервиса: `/health/live` враћа `Healthy` без обзира на доступност зависности; `/health/ready` враћа `Healthy + 200` када је зависност доступна, а `Unhealthy + 503` са читљивом поруком грешке када није.
- `ObservabilityPipelineTests` (сва четири сервиса) — тестира да наш код исправно региструје и активира OTel инструментацију, коришћењем уграђеног `System.Diagnostics.ActivityListener` (без OTel-специфичних тест пакета, без Docker зависности). Прва верзија теста (hook на `ActivityStarted`) није прошла — открила је стваран детаљ: `Activity.DisplayName` у тренутку `ActivityStarted` је сирово име извора (нпр. „Microsoft.AspNetCore.Hosting.HttpRequestIn”), не обогато име руте које се види у Tempo-у; обогатовање се дешава тек при `ActivityStopped`, када је позната рута. Исправка (hook на `ActivityStopped`) је и исправнија и тачније одражава оно што се стварно види у tracing backend-у.
- `TripClientTests (AuthService.Tests)` — регресиони тест за поменути „тих” catch блок. Са лажним `HttpMessageHandler`-ом (успех / HTTP 500 / баца `HttpRequestException`) и лажним `ILogger<TripClient>` који хвата позиве, три теста доказују да успешан позив не производи ниједан лог, неуспешан HTTP статус производи `LogWarning`, а мрежни квар производи `LogError`.

Резултати

Табела 6.1. Резултати аутоматизованог тестирања (dotnet test)

Пројекат	Passed	Failed	Total
TripService.Tests	6	0	6

AuthService.Tests	7	0	7
SharingService.Tests	4	0	4
Gateway.Tests	4	0	4
Укупно	21	0	21

Сви тестови пролазе. Тест пројекти су додати у TravelPlanner.sln, па се граде и приказују заједно са остатком решења у Visual Studio-у. Детаљан план и образложење обима налазе се у пратећем фајлу docs/testing.md.

[ПОПУНИ: убацити слику 6.1 — screenshot Visual Studio Test Explorer-a са свих 21 тестом зеленим (passed), ради директног визуелног доказа резултата из Табеле 6.1.]

Демонстрациони сценарији

Сва четири сценарија извршена су над стварно постављеним системом (Service Fabric локални кластер, праве базе), не симулирана — детаљна документација са тачним trace ID-јевима, log редовима и измереним временима налази се у пратећем фајлу docs/observability-scenarios.md; овде су сумирани кључни резултати.

Сценарио 1 — нормалан захтев кроз више сервиса

Пријава корисника, креирање VIEW дељења плана, па разрешавање тог кода као анониман посетилац — последњи корак намерно прелази три процеса: Gateway → SharingService → TripService. Добијен trace показује сервер span на Gateway-у, дете client span ка SharingService-у, унутар њега сервер span и даље дете client span ка TripService-у, а унутар TripService-овог сервер span-а девет SQL child span-ова (по један за сваку табелу коју InternalController.GetFullPlan чита). Сваки SQL упит добио је сопствени child span, исправно угнежђен под HTTP сервер span-ом који га је изазвао — аутоматски, без иједне линије custom кода за пропагацију контекста. Логови сва три сервиса корелисани су преко истог trace_id, укључујући и EF Core-ове сопствене „Executed DbCommand” записе са тачним трајањем по упиту.

[ПОПУНИ: убацити слику 6.2 — screenshot Grafana Explore -> Loki -> log ред овог истог захтева, са видљивим "traceid" пољем, ради приказа log-trace корелације описане изнад (упарити са Сликаом 5.3, исти trace).]

Сценарио 2 — грешка (над зависности)

TripService је рестартован кроз Service Fabric-ов сопствени механизам (Restart-ServiceFabricDeployedCodePackage) — покушај да се процес угаси директно са нивоа оперативног система је одбијен, јер сервис ради под другим Windows налогом него интерактивна сесија. Gateway-ов /health/ready је истог тренутка пријавио Unhealthy статус за trip-service проверу, систем се сам опоравио за приближно седам секунди, а health check инфраструктура је аутоматски забележила Error-ниво лог са тачном поруком узрока кvara — без иједне линије custom кода написане за ову конкретну ситуацију.

[ОПЦИОНО - ПОПУНИ: убацити слику 6.3 — screenshot одговора /health/ready БАШ у тренутку кvara, са "trip-service": "Unhealthy". Ово захтева тајминг уживо (позвати /health/ready одмах након рестарта, пре опоравка за ~7 секунди) - прескочи ако је незгодно да ухватиш тачан тренутак.]

Сценарио 3 — повећано оптерећење

k6 load test (скрипта: observability/load-tests/scenario3-load-test.js) са расподелом до 30 паралелних виртуелних корисника, реалистичан ток (пријава па преглед планова). Резултат: 780 захтева, 0% неуспешних, p95 латенција измерена на клијенту (k6) износила је 2.12 секунде, а независно измерена на серверу (Prometheus хистограм из OTel метрике) 2.14 секунде — готово идентично, што потврђује да метрике стварно одражавају понашање система под оптерећењем, не само да „нешто броје”.

[ПОПУНИ: убацити слику 6.4 — screenshot терминала са k6 излазом (780 захтева, 0% неуспешних, p95 латенција), ради доказа резултата Сценарија 3 наведеног изнад.]

Сценарио 4 — спора зависност (уско грло)

Директном SQL трансакцијом закључан је тачно један ред у табели Plans (X lock, без commit-a), а истовремено је упућен прави HTTP захтев (PUT /api/trips/{id}) који мења исти ред. Trace показује да је укупан захтев трајао 23.9 секунди, при чему SELECT упит у истом захтеву траје свега 14.5 милисекунди — дистрибуирано праћење захтева тачно показује да је узрок UPDATE наредба, не захтев уопштено. Успут је откривен и стваран налаз: TripsDB има укључен READ_COMMITTED_SNAPSHOT, па читања не чекају на закључан ред (потврђено директним увидом у sys.dm_tran_locks); тек упис са стварно измењеном вредношћу (writer-vs-writer сукоб, на који snapshot изолација не утиче) изазива право чекање.

[ОПЦИОНО - ПОПУНИ: убацити слику 6.5 — screenshot Grafana Explore -> Tempo waterfall-a за овај trace, са видљивим тачним трајањем SELECT (14.5ms) наспрам укупног захтева (23.9s), ради визуелног доказа да tracing прецизно локализује узрок.]

Табела 6.2. Поређење система пре и после надоградње

Област	Пре надоградње (AS-IS)	После надоградње (TO-BE)
Logging	Нула позива ILogger; подразумевани Console logger, ниво Information непопуњен садржајем.	Структурисани логови (Microsoft.Extensions.Logging + OTel provider) са аутоматским trace_id/span_id, слати ка Loki-ју преко OTLP.
Metrics	Не постоје ниједне метрике перформанси нити ресурса.	RED метрике + ресурси извршног окружења (CPU/меморија/GC), аутоматски из ASP.NET Core/HttpClient/Runtime инструментације.
Tracing	Не постоји; нема correlation/trace ID-а нигде у систему.	Пун W3C дистрибуирано праћење захтева кроз све сервисе, укључујући сервис-сервис позиве, аутоматска пропација без custom кода.
Monitoring	Нема визуелизације; једини увид је ручно читање конзолног излаза.	Grafana dashboard (RED метрике, ресурси, log stream) провисиониран и филтриран по сервису.
Health checks	Не постоје; Service Fabric нема увид у апликациони health.	/health/live и /health/ready на сва четири сервиса, укључујући проверу база и downstream зависности.

Error detection	Тихе грешке (нпр. TripClient catch блок без иједног лога).	Грешке производе log записе одговарајућег нивоа (Warning/Error), видљиве и корелисане са trace-ом.
Debugging	Ручно упоређивање временских печата логова из различитих процеса.	Један trace_id повезује логове, метрике и trace кроз сва три сервиса укључена у захтев.

Осим формалних тестова, решење је верификовано и кроз четири демонстрациона сценарија изведена над стварно постављеним системом (изнад), што заједно чини двослојну верификацију: аутоматизовани тестови доказују да наш код исправно ради у изолацији, а демонстрациони сценарији доказују да цео систем — укључујући Docker инфраструктуру, стварну базу података и стварну мрежну комуникацију између процеса — ради исправно у целини.

7. Закључак

У овом раду анализиран је постојећи систем Travel Planner са аспекта обсервабилности и идентификовани су следећи конкретни недостаци:

- одсуство дистрибуираног праћења захтева кроз више сервиса;
- одсуство метрика перформанси;
- „тихе“ грешке — примери у коду који гутају грешке без иједног трага у логу;
- одсуство health check механизма;
- нестандартизовано, минимално логовање.

Имплементирана је надоградња заснована на OpenTelemetry стандарду и Prometheus/Grafana Tempo/Grafana Loki/Grafana инфраструктури, без измене постојеће пословне логике.

Решење је верификовано на два независна начина: аутоматизованим тестовима (21 тест, сви пролазе, поглавље 6) и кроз четири демонстрациона сценарија изведена над стварно постављеним системом (нормалан захтев кроз три сервиса, намерна грешка са аутоматским опоравком, k6 load test са потврђеним поклапањем клијентске и серверске латенције, и намерно успорена SQL зависност чији је tracing тачно локализовао — не само да је захтев спор, већ и која конкретна SQL наредба је узрок).

Ограничења

Health check позиви (/health/live, /health/ready) пролазе кроз исту ASP.NET Core инструментацију као и сав остали саобраћај, па се њихови одговори мешају у исте метрике захтева као и стварни кориснички захтеви — за производни dashboard било би потребно експлицитно искључити /health/* путање из панела за стопу грешака.

Observability инфраструктура (Docker Compose стек) ради независно од постојећег Service Fabric кластера; интеграција са Service Fabric-овим сопственим Health Manager механизмом (тако да апликациони health утиче и на SF-ове одлуке о опоравку) није реализована у овом раду.

Distributed tracing тренутно почиње на Gateway-у; frontend (browser) није инструментисан OpenTelemetry Web SDK-ом, па је почетни, мрежни део захтева (од корисничког browser-а до Gateway-а) ван обухвата trace-а.

Правци даљег развоја

- Формално alerting решење (нпр. Prometheus Alertmanager) изнад постојећих Grafana dashboard-а, за проактивно обавештавање уместо ручног прегледа.
- Инструментација frontend-а (OpenTelemetry Web SDK) ради потпуног end-to-end trace-а, од browser-а до базе података.
- Аутоматизација k6 load test-а као дела CI/CD pipeline-а, ради континуираног праћења деградације перформанси између верзија.
- Интеграција апликационог health статуса са Service Fabric Health Manager-ом, ради дубље сарадње са платформом за опоравак од кvara.

Литература

- [1] IEEE, "IEEE Reference Guide." [ieeauthorcenter.ieee.org. https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf](https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf) (приступљено: септембар 2026).
- [2] OpenTelemetry Authors, "OpenTelemetry .NET Documentation." [opentelemetry.io. https://opentelemetry.io/docs/languages/net/](https://opentelemetry.io/docs/languages/net/) (приступљено: септембар 2026).
- [3] Microsoft, "Health checks in ASP.NET Core." [learn.microsoft.com. https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/health-checks](https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/health-checks) (приступљено: септембар 2026).
- [4] Microsoft, "Service Fabric documentation." [learn.microsoft.com. https://learn.microsoft.com/en-us/azure/service-fabric/](https://learn.microsoft.com/en-us/azure/service-fabric/) (приступљено: септембар 2026).
- [5] YARP contributors, "YARP: Yet Another Reverse Proxy - Documentation." [microsoft.github.io. https://microsoft.github.io/reverse-proxy/](https://microsoft.github.io/reverse-proxy/) (приступљено: септембар 2026).
- [6] Grafana Labs, "Grafana Tempo documentation." [grafana.com. https://grafana.com/docs/tempo/latest/](https://grafana.com/docs/tempo/latest/) (приступљено: септембар 2026).
- [7] Grafana Labs, "Grafana Loki documentation." [grafana.com. https://grafana.com/docs/loki/latest/](https://grafana.com/docs/loki/latest/) (приступљено: септембар 2026).
- [8] Grafana Labs, "Grafana k6 documentation." [grafana.com. https://grafana.com/docs/k6/latest/](https://grafana.com/docs/k6/latest/) (приступљено: септембар 2026).
- [9] Prometheus Authors, "Prometheus documentation." [prometheus.io. https://prometheus.io/docs/introduction/overview/](https://prometheus.io/docs/introduction/overview/) (приступљено: септембар 2026).
- [10] The OpenTelemetry Collector Authors, "OpenTelemetry Collector documentation." [opentelemetry.io. https://opentelemetry.io/docs/collector/](https://opentelemetry.io/docs/collector/) (приступљено: септембар 2026).

Биографија

Дајана Булић је рођена 2003. године у Врбасу. Завршила је електротехничку средњу школу Михајло Пупин у Новом Саду. Факултет техничких наука у Новом Саду уписала је 2022. године, на студијском програму Примењено софтверско инжењерство. Испунила је све обавезе и положила све испите предвиђене студијским планом и програмом.